1 : We built a mini Android/AOSP development environment using:

- **Gerrit** → Code review system
- **Git** → Version control
- **repo tool** → Manage multiple repositories
- **Manifest repository** → Define all projects

Real Android companies use this architecture.

Typical real workflow:

```
Developer
    ↓
repo upload
    ↓
Gerrit Code Review
    ↓
Jenkins CI
    ↓
Build/Test
    ↓
Merge
```

2. Install and Setup Gerrit:

You first installed Gerrit on your Linux machine.

Gerrit runs on:

Web UI → http://localhost:8080
SSH → port 29418

Check Gerrit SSH connection:

ssh -p 29418 hemant11ks@localhost

Output:

Welcome to Gerrit Code Review

This confirms SSH authentication works.

3. SSH Authentication Issue:

Problem

When cloning the repository you saw:

Permission denied (publickey)

Reason

SSH key was not configured or wrong username was used.

Linux user: **hemant11**

Gerrit user: **hemant11ks**

Fix

Use correct Gerrit username:

git clone ssh://hemant11ks@localhost:29418/manifest

4. Create Manifest Repository:

In Gerrit UI you created:

manifest

Purpose:

The manifest repository defines **which repositories belong to the workspace**.

5. Clone Manifest Repository:

1. Create workspace:

cd ~

mkdir workspace

cd workspace

2. Clone manifest repo:

git clone ssh://hemant11ks@localhost:29418/manifest

cd manifest

Structure:

workspace/

└── manifest

6. Install Gerrit commit-msg Hook:

Gerrit requires a **Change-Id** in commit messages.

Install hook:

curl -Lo .git/hooks/commit-msg http://localhost:8080/tools/hooks/commit-msg

chmod +x .git/hooks/commit-msg

Without this hook, Gerrit rejects commits.

7. Create Manifest File:

Create folder:

mkdir manifests

Create file:

nano manifests/default.xml

Example manifest:

```
<manifest>

 <remote name="origin"
        fetch="ssh://hemant11ks@localhost:29418"/>

 <default remote="origin"
         revision="master"/>

 <project name="android-demo"
         path="android-demo"/>

</manifest>
```

Explanation:

Element	Purpose
remote	Gerrit server
default	default branch
project	repositories to fetch

8. Push Manifest to Gerrit:

- git add .
- git commit -m "Add default manifest"
- git push origin HEAD:refs/for/master

Important:

refs/for/master

This sends the change **for Gerrit review**, not directly to master.

Then approve and **Submit** in Gerrit UI.

9. Initialize Repo Workspace:

Create Android workspace:

- **cd ~/workspace**

mkdir android-workspace

- **cd android-workspace**

Initialize repo:

- **repo init -u ssh://hemant11ks@localhost:29418/manifest -m manifests/default.xml**

This creates:

```
android-workspace/
└── .repo
```

`.repo` stores repo metadata.

10. Run repo sync:

- **repo sync**

This clones repositories listed in the manifest.

Workspace becomes:

```
android-workspace/
├── .repo
└── android-demo
```

11. Issue: SSH Username Error:

Error:

Permission denied (publickey)

Repo tried:

ssh://localhost

which used Linux username:

hemant11

Fix : Edit manifest:

fetch="ssh://hemant11ks@localhost:29418"

Push update to Gerrit.

Then run:

- **repo sync**

12. Issue: .repo Folder Added to Git:

Warning:

adding embedded git repository

Reason:

- **git add .**

added `.repo`.

Fix:

- **git rm -r --cached .repo**

Add to `.gitignore`:

.repo/

Then amend commit:

- **git commit --amend**

13. Create Second Repository:

To simulate multiple repos we created:

service-demo

Clone it:

- **git clone ssh://hemant11ks@localhost:29418/service-demo**
- **cd service-demo**

Add file:

- **echo "service code" > service.txt**
- **git add .**
- **git commit -m "Initial service repo"**

14. Issue: Missing Change-Id:

Error:

missing Change-Id in message footer

Fix:

Install hook:

- **curl -Lo .git/hooks/commit-msg http://localhost:8080/tools/hooks/commit-msg**
- **chmod +x .git/hooks/commit-msg**

Amend commit:

- **git commit --amend --no-edit**

Push again:

- **git push origin HEAD:refs/for/master**

15. Add Feature Change:

Modify file:

- **echo "new feature" >> service.txt**
- **git add .**
- **git commit -m "Add new feature"**

Push to Gerrit:

- **git push origin HEAD:refs/for/master**

16. Repo Workspace Behavior:

When you ran:

- **repo forall -c "git branch"**

Output:

* (no branch)

Reason:

Repo sync checks out a specific commit (detached HEAD).

17. Create Development Branch:

Create branch across all repos:

- **repo start feature1 --all**

Verify:

- **repo forall -c "git branch"**

Output:

* feature1

18. Useful Repo Commands:

Check status across repos:

- **repo status**

Run command in all repos:

- **repo forall -c "git status"**

Show repo names:

- **repo forall -c "echo $REPO_PROJECT"**

19. Repo Workspace Structure:

Final structure:

```
workspace/
├── manifest
├── android-workspace
│   ├── .repo
│   ├── android-demo
│   └── service-demo
```

---

20. Real Android Workflow

Actual developer workflow:

- **repo sync**
- **repo start feature1 --all**

edit code

git commit

repo upload

Then Gerrit handles review.

---

21. What You Learned

You now know:

- Gerrit code review workflow
- SSH authentication
- commit-msg hook
- manifest repositories
- repo init

- repo sync
- multi-repository workspace
- repo start
- pushing changes to Gerrit

This is **very close to real Android platform development**.

---

22. Real Industry Use Case

Large companies like:

- Google
- Samsung
- Qualcomm
- OnePlus

manage **hundreds of repositories** using this architecture.

Example:

frameworks/base

system/core

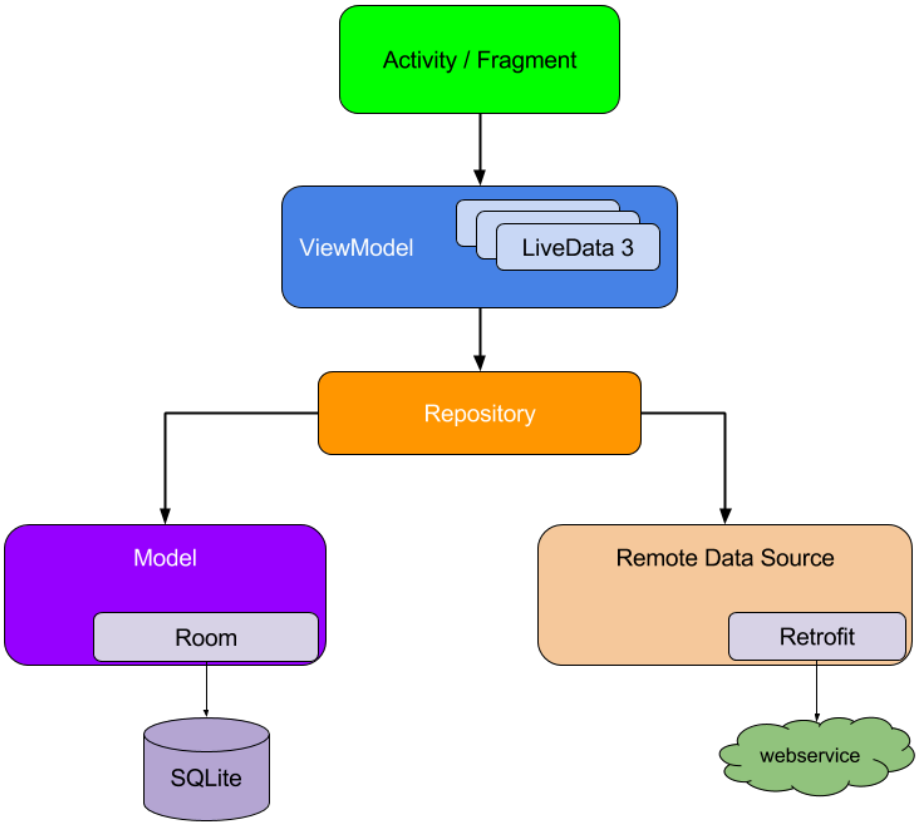
packages/apps

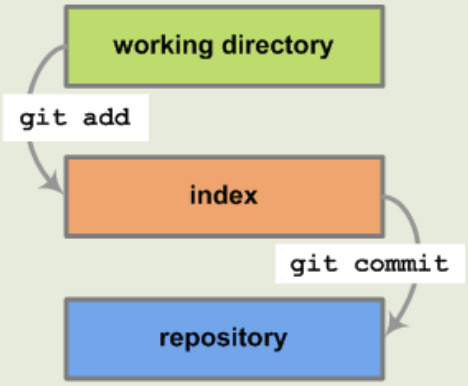
hardware/interfaces

device/google

All managed via **manifest + repo tool**.

THEORY :


Activity / Fragment
ViewModel
LiveData 3
Repository
Model
Room
Remote Data Source
Retrofit
SQLite
webservice



Edit the files using vim / emacs / etc.,

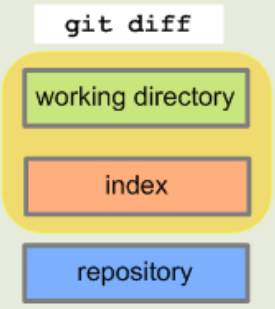
Stage the changes using `git add`

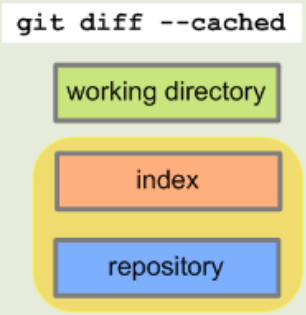
Review the changes using `repo status` .

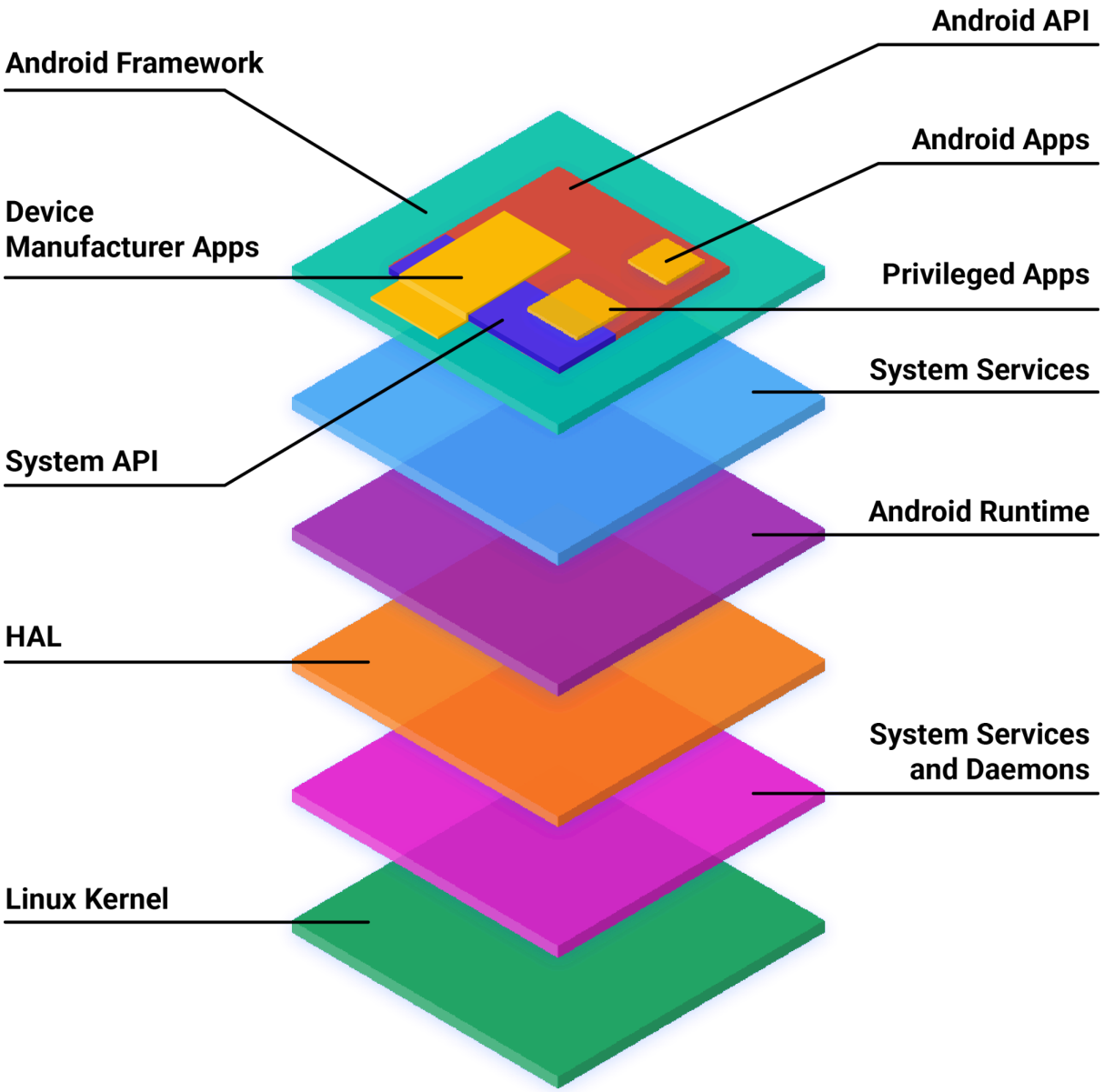
Commit the changes using `git commit`

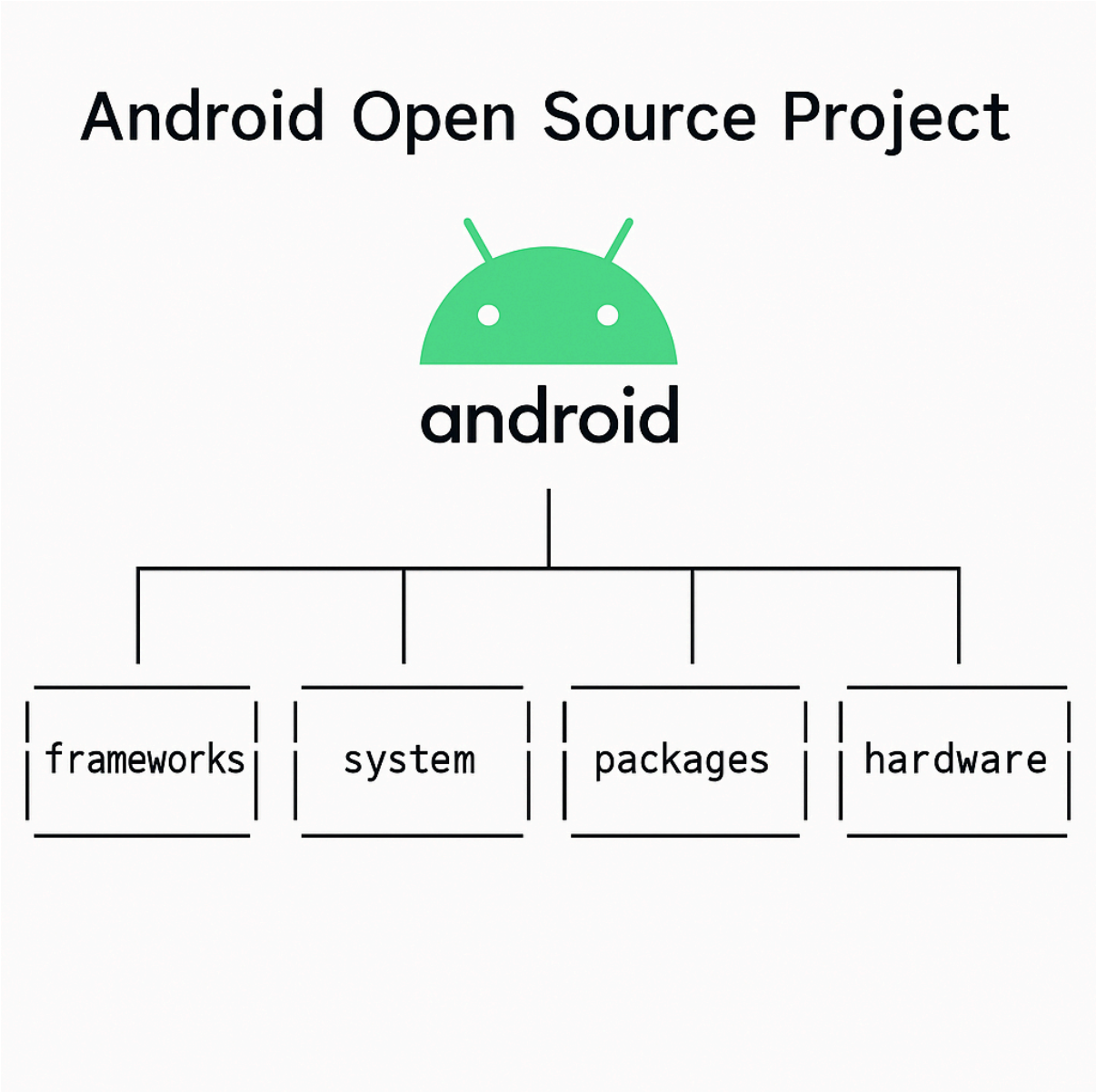
Common Commands

`repo init`	initializes a new client
`repo sync`	syncs client to repositories
`repo start`	starts a new branch
`git add`	stages files
`repo status`	shows status of current branch
`git commit`	commits staged files
`git branch`	shows current branches
`git branch [branch]`	creates new topic branch
`git checkout [branch]`	switches HEAD to specified branch
`git merge [branch]`	merges [branch] into current branch
`git diff`	shows diff of unstaged changes
`git diff --cached`	shows diff of staged changes
`git log`	shows history of current branch
`git log m/[codeline]..`	shows commits that are not pushed
`repo upload`	uploads changes to review server






Android API
Android Framework
Android Apps
Device
Manufacturer Apps
Privileged Apps
System Services
System API
Android Runtime
HAL
System Services
and Daemons
Linux Kernel



Components in the Architecture

Component	Purpose
Gerrit	Code review and Git server
Manifest Repository	Defines which repositories belong to the workspace
Repo Tool	Manages multiple Git repositories
Project Repositories	Actual codebases (android-demo, service-demo)
Developer Workspace	Local development environment

2Manifest Repository Concept:

Instead of cloning many repos manually, Android uses one manifest file.

Example we created:

```
<manifest>

  <remote name="origin"
        fetch="ssh://hemant11ks@localhost:29418"/>

  <default remote="origin"
         revision="master"/>

  <project name="android-demo"
         path="android-demo"/>

  <project name="service-demo"
         path="service-demo"/>

</manifest>
```

Meaning:

Repository	Local Directory
android-demo	android-demo/
service-demo	service-demo/

3Repo Initialization Workflow:

Command used:

- **repo init -u ssh://hemant11ks@localhost:29418/manifest -m manifests/default.xml**

What happens internally:

```
repo init
     |
     ├── clone manifest repo
     |
     ├── read default.xml
     |
     └── create .repo directory
```

Workspace structure:

```
android-workspace/
 ├── .repo
 └── manifest data
```

4Repository Sync Workflow:

Command:

- **repo sync**

Repo reads `default.xml` and clones repositories automatically.

Result:

Result:

```
android-workspace/
 ├── .repo
 ├── android-demo
 └── service-demo
```

This replaces manually running:

```
git clone repo1
git clone repo2
git clone repo3
```

5 Developer Workflow:

Typical Android developer workflow:

- **repo sync**
- **repo start feature1 --all**

Then developers modify code:

- **cd service-demo**

edit code

- **git add .**
- **git commit**

6 Gerrit Code Review Workflow:

When pushing changes:

- **git push origin HEAD:refs/for/master**

or

repo upload

Gerrit creates a **change review**.

```
Developer
    |
    └─ push refs/for/master
           |
           ▼
        Gerrit
           |
     Code Review
           |
        Approve
           |
        Submit
           |
        Merge
```

7Patchset Workflow (Important Gerrit Concept):

When you update the same change:

- **git commit --amend**
- **git push origin HEAD:refs/for/master**

Gerrit creates a **new patchset**.

Example:

Change 8

├─ Patchset 1

├─ Patchset 2

└─ Patchset 3

This lets reviewers track revisions.

---

8 Repo Workspace Behavior

When running:

- **repo forall -c "git branch"**

You saw:

(no branch)

Reason:

Repo checks out **specific commits**, not branches.

To start development:

- **repo start feature1 --all**

Now each repository gets:

feature1 branch

9Useful Repo Commands:

Check status across repositories

- **repo status**

Run command in all repositories

- **repo forall -c "git status"**

Show repo names

- **repo forall -c "echo $REPO_PROJECT"**

Show last commit in every repo

- **repo forall -c "git log -1 --oneline"**

10. Real Android Use Case:

Large Android codebases contain **hundreds of repositories**.

Example AOSP structure:

```
platform/
├── frameworks/base
├── system/core
├── packages/apps/Settings
├── hardware/interfaces
└── device/google/pixel
```

Each is a separate Git repository, managed by repo + manifest.

11 Full Workflow Summary:

```
Create Gerrit Server
        |
Create Manifest Repository
        |
Define Repositories in default.xml
        |
repo init
        |
repo sync
        |
Multi-repository workspace created
        |
repo start feature
        |
Developer edits code
        |
git commit
        |
repo upload
        |
Gerrit review
        |
Merge
```

12Skills You Have Now:

You practiced:

✔ Gerrit code review
✔ SSH authentication
✔ commit-msg hook
✔ manifest repositories
✔ repo init
✔ repo sync
✔ multi-repository workspace
✔ repo start
✔ pushing changes to Gerrit

These are core Android platform development skills.

1 What is the `.repo` Directory:

When you ran:

- **repo init**

repo created this structure:

android-workspace/

├── .repo

├── android-demo

└── service-demo

`.repo` is not a normal folder.
It stores all metadata required to manage multiple Git repositories.

Think of `.repo` as:

.repo = repo tool brain

---

2 Structure of `.repo`:

Your directory currently contains:

.repo/

├── manifests

├── manifests.git

├── manifest.xml

├── repo

└── TRACE_FILE

Let's explain each.

---

manifests.git

.repo/manifests.git

This is the actual Git clone of the manifest repository.

Meaning repo internally ran:

git clone ssh://gerrit/manifest

This repo stores:

default.xml

---

manifests

.repo/manifests

This is the working copy of the manifest repository.

Example:

.repo/manifests/

└── manifests/default.xml

Repo reads this file when running:

repo sync

---

manifest.xml

.repo/manifest.xml

This is the resolved manifest file used by repo.

Repo combines:

default.xml

local manifests

into one final manifest.

Example:

- **<manifest>**
- **<project name="android-demo" path="android-demo"/>**
- **<project name="service-demo" path="service-demo"/>**
- **</manifest>**
- **Repo uses this file during sync operations.**

---

repo

.repo/repo

This folder contains the repo tool source code.

Example:

```
.repo/repo/
├── main.py
├── subcmds/
└── project.py
```

This is why repo works even if your system doesn't install it globally.

TRACE_FILE

.repo/TRACE_FILE

Used internally by repo for debugging and tracing operations.

Developers rarely interact with it.

---

3 Hidden Repo Git Objects

In large workspaces repo also creates:

.repo/project-objects

.repo/projects

These optimize Git performance when working with hundreds of repositories.

Example:

.repo/project-objects/android-demo.git

This stores Git objects shared across clones.

---

4 How `repo sync` Works Internally

When you run:

repo sync

repo performs:

Step 1

Read .repo/manifest.xml

Step 2

Check which repositories are listed

Step 3

Clone missing repos

Step 4

Fetch updates for existing repos

Step 5

Checkout exact commit

That is why your repositories appeared in detached HEAD state.

---

5 Debugging Repo Problems

When repo sync fails, Android engineers check:

cat .repo/manifest.xml

to confirm which repositories should be cloned.

Example:

repo status

repo sync -j1 --fail-fast

These commands help debug failures.

---

6 Useful Repo Debug Commands

Show manifest repo

repo manifest

Show all projects

repo list

Example output:

android-demo : android-demo

service-demo : service-demo

Run command across repos

repo forall -c "git status"

Show current manifest

repo manifest -r

---

7 Why Repo Was Created

Google created repo because Git alone cannot efficiently manage hundreds of repositories.

Example AOSP repo count:

800+ repositories

Without repo developers would need:

git clone repo1

git clone repo2

git clone repo3

...

git clone repo800

Repo simplifies this to:

repo init

repo sync

---

8 Real AOSP Workspace

Typical Android developer workspace:

```
~/aosp/
├── .repo
├── frameworks
├── system
├── packages
├── build
└── hardware
```

All controlled by `.repo`.

---

9 Key Commands You Learned

repo init

repo sync

repo start

repo status

repo forall

repo upload

These are core Android platform development commands.

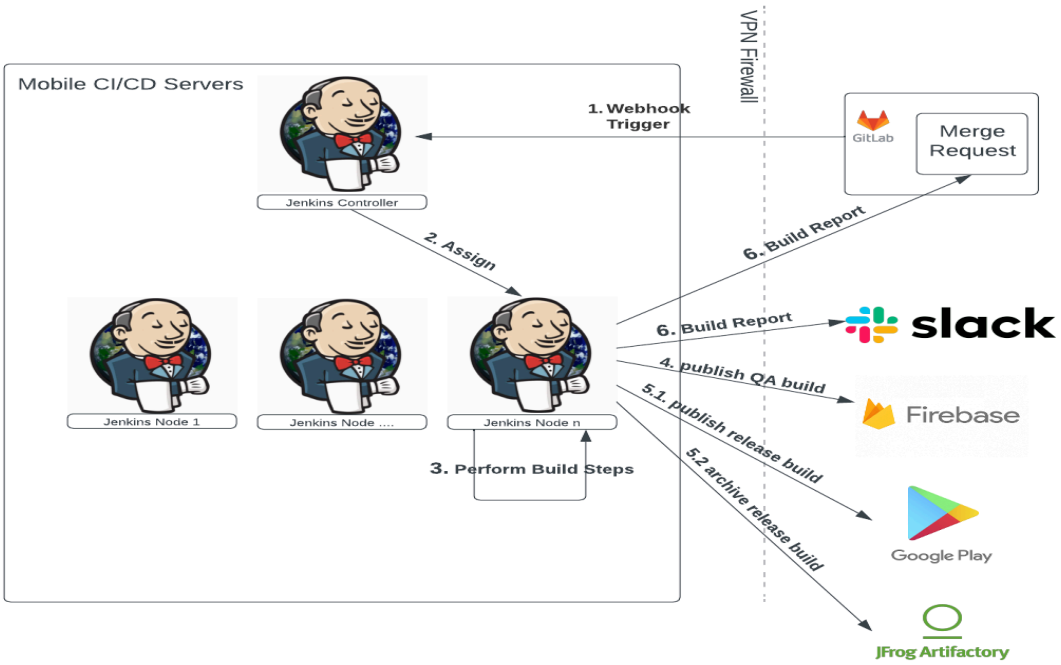
🔟 Your Current Skill Level

You now understand:

- **Gerrit code review**
- **repo init / repo sync**
- **manifest repository**
- **multi-repository workspace**
- **repo start workflow**
- **Gerrit patchsets**
- `.repo` **directory internals**

This is exactly how Android platform teams work.

CI/CD using Jenkins + Gerrit:



Components

Component	Purpose
Developer	Writes code
Repo Tool	Uploads changes
Gerrit	Code review system
Jenkins	CI server that builds/tests
Build System	Compile code / run tests

2 Real Development Workflow

When a developer uploads code:

repo upload

Gerrit receives the patch.

Example change:

service-demo

Change 8

Patchset 1

Then Gerrit triggers Jenkins.

---

3 CI Pipeline Flow

Full pipeline:

- **Developer**
- |
- **repo upload**
- |
- **Gerrit**
- |
- **Trigger Jenkins Job**
- |
- **repo sync**
- |
- **Build Project**
- |
- **Run Tests**
- |
- **Send Result to Gerrit**

---

4 Jenkins Job Process

Typical Jenkins job does this:

Step 1 — Clone Workspace

repo init -u ssh://gerrit.company.com/platform/manifest

repo sync

Step 2 — Fetch Patchset

Jenkins fetches the patch under review.

Example:

git fetch origin refs/changes/08/8/1

git checkout FETCH_HEAD

This builds the **exact change under review**.

Step 3 — Build Project

Example build commands:

make

or for Android:

source build/envsetup.sh

lunch aosp_arm64

make -j16

Step 4 — Run Tests

Example:

pytest

or

cts test

Step 5 — Report Result to Gerrit

Jenkins sends vote back to Gerrit.

Example:

Verified +1

Meaning build passed.

If build fails:

Verified -1

---

5 Gerrit Review Labels

Typical Gerrit labels:

Label	Meaning
Code-Review +2	**Approved by reviewer**
Verified +1	**Build passed**
Verified -1	**Build failed**

Example:

- **Change 8**
- **Code-Review +2**
- **Verified +1**
- **Now change can be merged.**

---

6 Example CI Pipeline

Example Jenkins pipeline:

- **pipeline {**
- **agent any**
-
- **stages {**
-
- **stage('Checkout') {**
- **steps {**
- **sh 'repo sync'**
- **}**
- **}**
-
- **stage('Build') {**
- **steps {**
- **sh 'make'**
- **}**
- **}**
-
- **stage('Test') {**
- **steps {**
- **sh 'pytest'**
- **}**
- **}**
-
- **}**
- **}**

---

7 Real Android CI

Android companies run massive CI systems.

Example pipeline:

- **repo upload**
- |
- **Gerrit**
- |
- **Jenkins Trigger**
- |
- **repo sync**
- |
- **Build AOSP**
- |
- **Run CTS Tests**

- |
- **Send Verified +1**

This ensures **bad code never reaches master branch**.

---

8 Why Gerrit is Used Instead of GitHub

Large companies prefer Gerrit because it supports:

Feature	Reason
Patchsets	Track revisions
Mandatory code review	Prevent bad merges
CI integration	Automated builds
Multi-repo support	Works with repo tool

---

9 Full System You Built

You recreated this architecture locally:

- **Gerrit**
- |
- **Manifest Repo**
- |
- **repo init**
- |
- **repo sync**
- |
- **android-demo**

- **service-demo**
- │
- **git commit**
- │
- **git push refs/for/master**
- │
- **Gerrit review**
- **Next step would normally be Jenkins CI trigger.**

🔟 What You Learned

You now understand:

- **Gerrit code review workflow**
- **manifest repositories**
- **repo init / repo sync**
- **multi-repository workspace**
- **Gerrit patchsets**
- **repo start / repo upload**
- **CI pipeline architecture**

These are **core DevOps + Android platform engineering skills**.